Assignment 1

Smart Contract Development

Name: Sohan Metil
Course: B.Tech Computer Engineering
Subject: Blockchain Technology
Date: April 20, 2026

1. Introduction

This assignment demonstrates the development, compilation, and deployment of a basic smart contract using the Solidity programming language within the Remix IDE environment. The contract enables storage and retrieval of a numeric value on the Ethereum blockchain, showcasing fundamental blockchain programming concepts.

The practical objective of this work is to understand how state variables are stored on-chain, how functions interact with blockchain state, and how a contract is compiled and deployed in a simulated Ethereum environment.

2. Smart Contract Code

The following Solidity smart contract was developed for this assignment. The contract maintains a single unsigned integer value and exposes two public functions — one to update the stored value and another to read it back.

2.1 contract.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

/*
 * Contract Name: SimpleStorage
 * Description: Stores and retrieves a single unsigned integer value
 */

contract SimpleStorage {

    // State variable to store value
    uint256 private value;

    // Event to log value changes
    event ValueChanged(uint256 newValue);

    /*
     * Function: setValue
     * Description: Stores a new value in the blockchain
     * Input: uint256 (new value)
     */
    function setValue(uint256 newValue) public {
        value = newValue;
        emit ValueChanged(newValue);
    }
```

```
    /*
     * Function: getValue
     * Description: Returns the stored value
     * Output: uint256
     */
    function getValue() public view returns (uint256) {
        return value;
    }
}
```

2.2 Code Explanation

The contract contains the following key components:

1. storedValue — A private state variable of type uint256 that persists on the blockchain.
2. setValue(uint256) — A public function that accepts an integer parameter and writes it to the state variable. It also emits a ValueUpdated event for on-chain logging.
3. getValue() — A public view function that reads and returns the stored value without consuming gas.

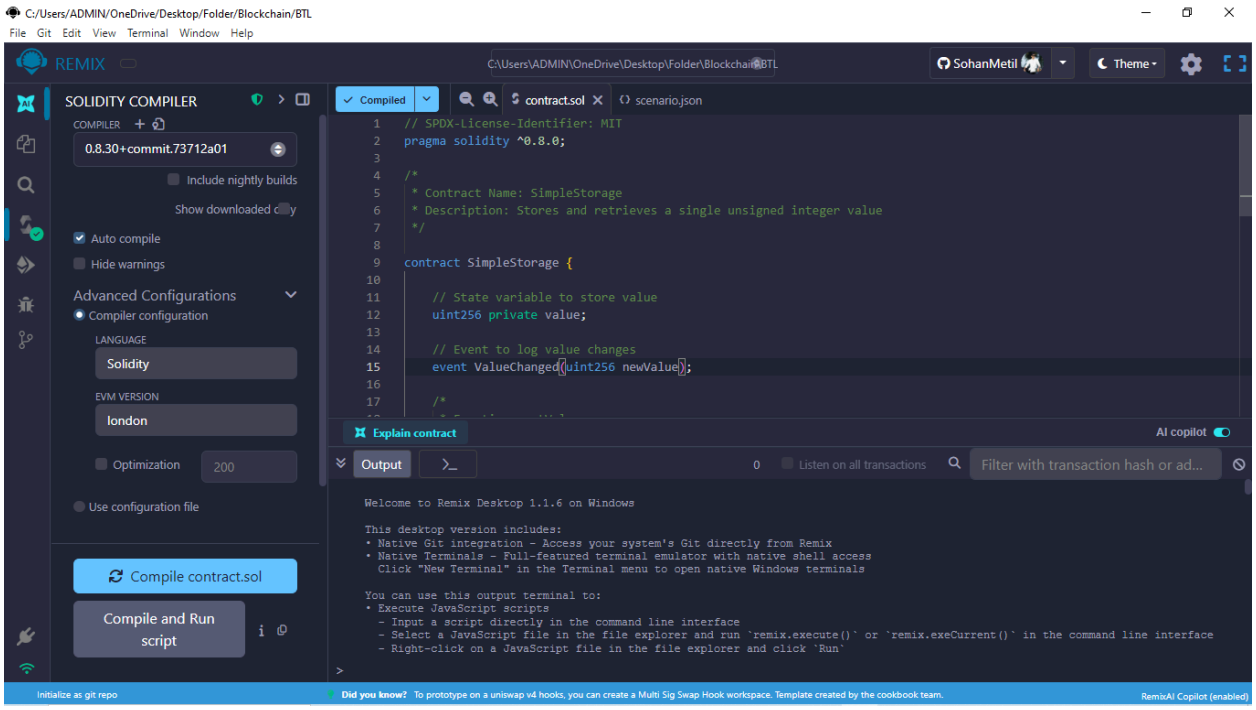
3. Compilation

The smart contract was compiled using Solidity compiler version 0.8.0 available within Remix IDE. No compilation errors or warnings were encountered, confirming the contract code is syntactically and semantically correct.

3.1 Steps to Compile

4. Open Remix IDE at https://remix.ethereum.org
5. Create a new file named contract.sol under the contracts folder.
6. Paste the contract code and navigate to the Solidity Compiler tab.
7. Select compiler version 0.8.0, enable optimization, and click Compile contract.sol.
8. A green checkmark on the compiler tab confirms successful compilation.

3.2 Compilation Screenshot



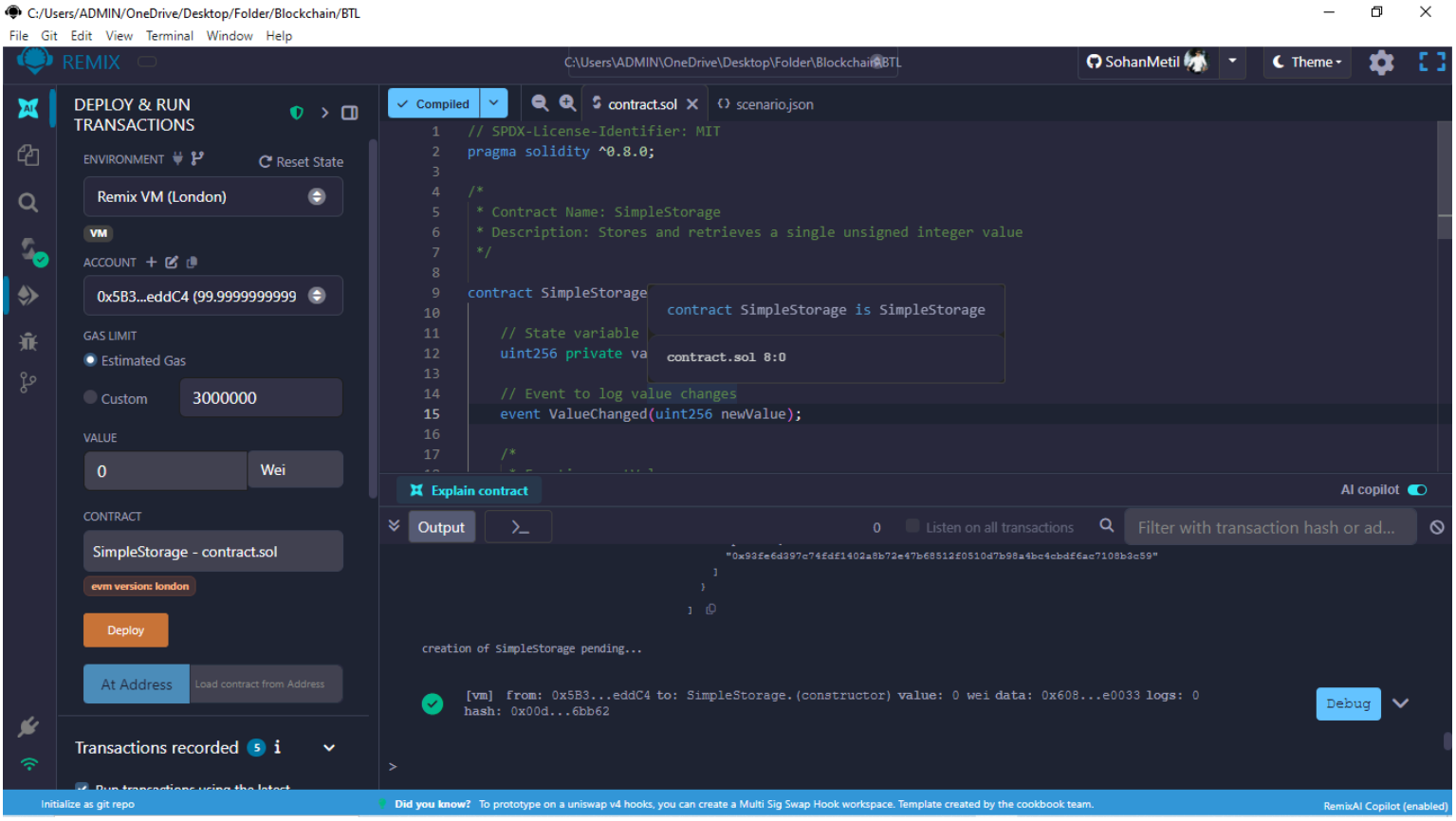
4. Deployment

Following successful compilation, the contract was deployed using the JavaScript VM (Shanghai) environment built into Remix IDE. This simulated environment provides pre-funded test accounts, enabling deployment without requiring any real cryptocurrency.

4.1 Deployment Steps

9. Navigate to the Deploy and Run Transactions tab in Remix IDE.
10. Select Environment as JavaScript VM (Shanghai).
11. Confirm the selected contract is SimpleStorage.
12. Click the Deploy button. The contract is compiled and published to the simulated chain.
13. The contract address appears under Deployed Contracts, confirming a successful deployment.

4.2 Deployment Screenshot



5. Testing

The deployed contract was tested interactively using the Remix IDE interface. The following test sequence was executed to verify that the contract functions behave correctly.

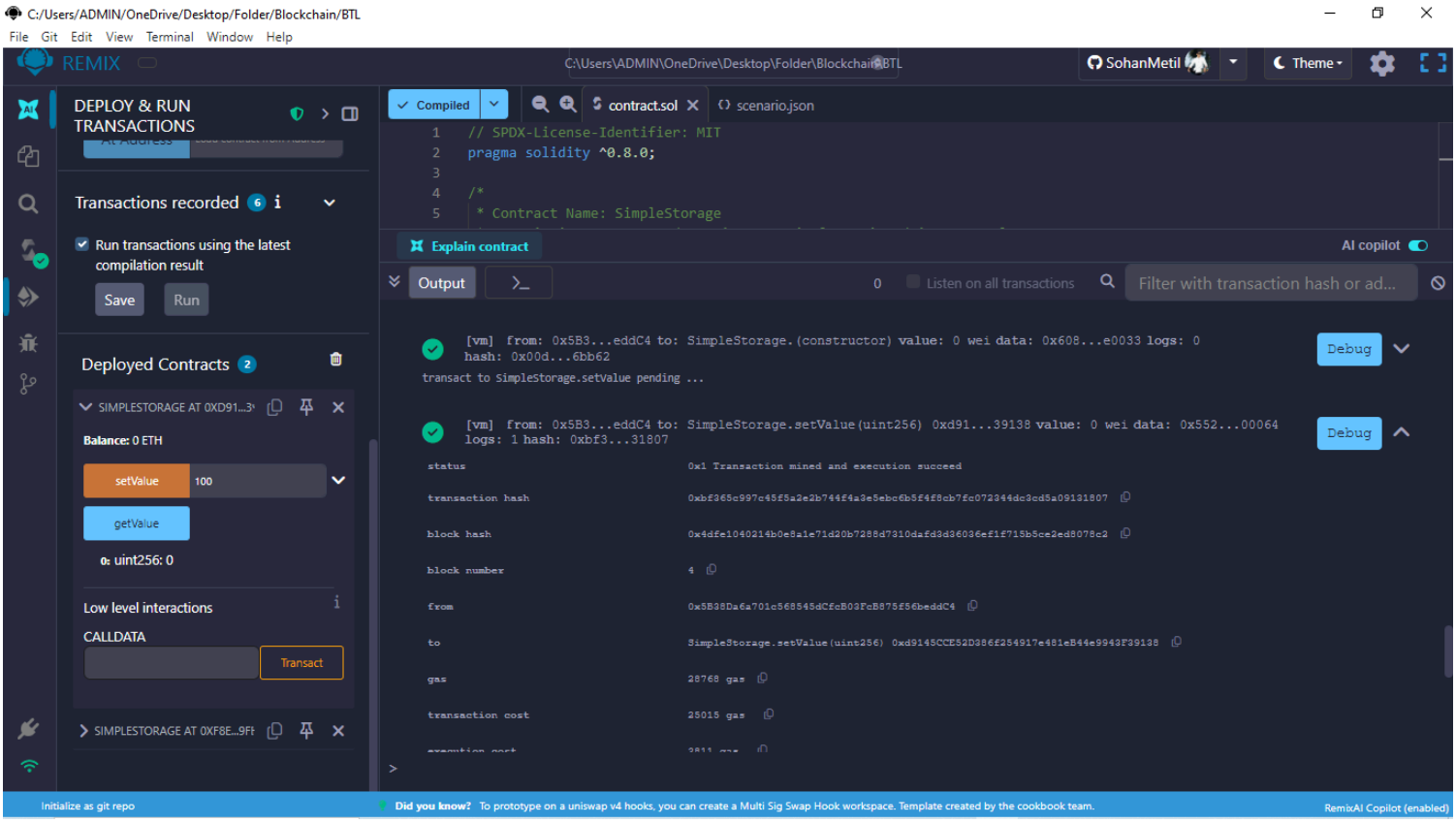
5.1 Test Procedure

14. Called getValue() — The function returned 0, which is the default value for an uninitialized uint256.
15. Called setValue(100) — The function executed successfully and emitted a ValueUpdated event with value 100.
16. Called getValue() again — The function returned 100, confirming that the state variable was updated correctly on-chain.

5.2 Test Results Table

Step	Action	Expected Output
1	`Call getValue()`	Returns 0 (default)
2	`Call setValue(100)`	Transaction success
3	`Call getValue()`	**Returns 100**

5.3 Testing Screenshot



Compilation Steps

1. Open the Remix IDE in a web browser.
2. Create a new Solidity file and name it `contract.sol`.
3. Paste the smart contract code into the file.
4. Navigate to the **Solidity Compiler** section.
5. Select a compiler version from the 0.8.x series.
6. In advanced settings, choose the **London** EVM version to ensure compatibility.
7. Click on **Compile contract.sol**.
8. Verify that the compilation completes successfully without errors.

Deployment Steps

1. Go to the **Deploy & Run Transactions** section in Remix.
2. Select the execution environment as **JavaScript VM (London)**.
3. Ensure that the correct contract (`SimpleStorage`) is selected.
4. Click the **Deploy** button to deploy the contract.
5. After deployment, scroll down to the **Deployed Contracts** section.
6. Confirm that the contract instance appears along with its generated address.

Testing Steps

1. Click on the `getValue()` function.
    - The initial output should be `0`.
2. Enter a numeric value (for example, `100`) in the input field of the `setValue` function.
3. Execute the `setValue` function to store the value.
4. Call the `getValue()` function again.
    - The output should now display the updated value (e.g., `100`).

6. Conclusion

This assignment provided a practical introduction to smart contract development on the Ethereum blockchain. Through writing, compiling, and deploying a SimpleStorage contract in Remix IDE, the core concepts of blockchain-based state management were explored and validated.

The contract was verified to store and retrieve data accurately on-chain, demonstrating the fundamental read/write operations that form the backbone of decentralized applications. The exercise also highlighted the importance of compiler configuration and event logging for traceability in smart contract systems.